A Double Deep Q-Network Reinforcement Learning Method for Load-Balancing Inter-Satellite Routing in LEO Satellite Constellations

Yiwei Chen, Haisu Zhang*, Qingsong Li, Liangqin Yu, Xin Zhou, Fengwu Li
Information Support Force Engineering University
Wuhan, China
*zhanghaisu@nudt.edu.cn

2026 6th International Conference on Intelligent Communications and Computing (ICICC) | 979-8-3195-3487-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICICC71012.2026.11638046

***Abstract*—Inter-satellite routing is essential for LEO satellite network reliability and traffic balancing. Conventional shortest-path routing causes local overload due to neglected link congestion, while existing RL routing approaches suffer from slow convergence, Q-value overestimation and non-standard interface designs. This paper presents a Double DQN-based load-balancing inter-satellite routing method. Taking node information and global topology-load states as inputs, it outputs forwarding directions mapped to satellite IDs. To handle large dynamic state space, multi-objective reward design and Q-value overestimation, we employ a dual-network framework, a distance-load joint reward, a dynamic ε-greedy strategy, and standardized satellite numbering and mapping rules. Experiments on a 66-satellite EXata simulation platform demonstrate that the method obtains favorable performance: 8 average hops, 270 ms maximum convergence time, 4.0% normalized overhead, 0.96 s end-to-end delay and 10% ground station traffic difference. The packet loss rate is only 0.3% under 20% satellite failure, fully meeting design requirements.**

Keywords—LEO satellite network, inter-satellite routing, load balancing, Double DQN, deep reinforcement learning

I. Introduction

LEO constellations contain numerous satellites operating on multiple orbital planes. Constant satellite movement brings time-varying topologies, frequent inter-satellite link changes and dynamically fluctuating link loads. Such characteristics empower LEO systems to provide global coverage for space-air-ground integrated networks and space-based IoT, while imposing high requirements on routing mechanisms. With uneven spatio-temporal traffic, inter-satellite routing determines data transmission reliability, load balancing and core network performance indicators including delay, packet loss and throughput.

Existing shortest-path routing is vulnerable to local overload and congestion [1,2]. DRL represented by DQN [3] has been widely applied to satellite routing, and GNN is often combined to adapt to dynamic topologies [2,4,5]. However, single-network DRL routing schemes suffer from Q-value overestimation, slow convergence and unstable training [6]. They also lack standardized satellite numbering, unified input/output designs and direction-ID mapping, which limits practical deployment. Related studies cover GNN-DRL routing [2,4], multi-agent DQN optimization [7], two-phase adaptive routing [5], DRL-based computation offloading and DDQN-based bandwidth allocation [8,9], all verifying the value of dual-network structures in space network decision-making.

This paper presents a DDQN-based load-balancing inter-satellite routing method for LEO constellations. It takes node information and global topology-load states as inputs, outputs forwarding directions and maps them to satellite IDs, to implement intelligent routing under complex network conditions. However, there exist three key challenges: huge state space caused by dynamic networks, contradictory routing objectives requiring trade-off between path length and load, and Q-value overestimation of traditional DQN that degrades real-time on-board routing performance.

Our main contributions are: (1) A dual-network architecture combined with Double DQN to eliminate Q-value overestimation and optimize convergence and stability; (2) A distance-load joint reward mechanism and dynamic ε-greedy strategy for congestion avoidance and exploration-exploitation balance; (3) Standardized satellite numbering, input format and direction-ID mapping to facilitate simulation and on-board application.

Experimental results on a 66-satellite HLA/RTI + EXata platform show that the method has 8 average hops, 270 ms maximum convergence time, 4.0% normalized overhead, 0.96 s end-to-end delay and 10% ground-station traffic difference. The packet loss rate is 0.3% under 20% satellite failure, satisfying all design targets.

II. System Model and Problem Formulation

This work considers a LEO Walker constellation with M orbital planes and N satellites per orbit, where satellites are numbered by orbit and in-orbit sequence. Each satellite carries four inter-satellite laser links connecting to its four adjacent satellites. User traffic is imported through access networks, and inter-satellite routing delivers data to ground stations for downlink.

Each satellite stores a global state table for node connectivity and link load. Connectivity is marked as 1 (connected) or MAX (disconnected), and link congestion is marked as 1 when queue usage exceeds 50% capacity and 0 otherwise. The table is synchronized in real time by the routing protocol and serves as the input for routing decisions.

Satellite motion causes dynamic topology changes, and each satellite only perceives local neighbor states. Packet forwarding

is a hop-by-hop sequential decision process, where each choice influences current delay and future network congestion. This characteristic matches the properties of Markov Decision Process (MDP). We model the load-balancing inter-satellite routing as an MDP and solve the optimal forwarding policy via reinforcement learning.

The system state combines service information (source, destination, current node) and network information (binary topology, link load), forming an integrated state vector for comprehensive routing judgment. The action space includes four discrete forwarding directions corresponding to four laser links. Using directions as actions rather than node IDs makes the action space independent of constellation size and eases model training.

After each forwarding action, the packet arrives at the next node, and link queue loads are updated to form a new stochastic state dominated by topology and traffic dynamics. The reward function encourages packets to approach the destination, punishes selections of congested links, and gives the highest reward upon successful delivery (see Section 3.4). The proposed policy aims to reduce end-to-end hops and delay, balance network load and avoid congestion, guaranteeing reliable transmission and service quality under complex operating conditions.

III. Double-DQN-Based Load-Balancing Inter-Satellite Routing Method

A. Overall Architecture

The proposed method adopts a Double DQN architecture composed of an estimation network and a target network that share an identical structure. During training, the estimation network learns iteratively in real time while the target network stably fits the Q-values; after training converges, the target network is deployed on the satellite nodes as the operational on-board model for real-time routing decisions. The model takes the business parameters (source address, destination address, current node) and the network state (topology and load information) as input, and outputs the next-hop forwarding direction of the data packet, as shown in Fig. 1.

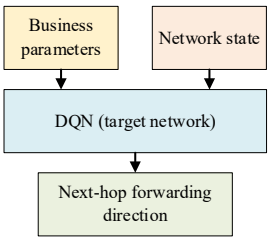

Fig. 1. Load-balancing algorithm model.

B. State-Space Design

The algorithm input comprises business information and node-state parameters. The business information contains the source, destination, and current node positions of the data packet; because the orbits and intra-orbit satellites of the Walker constellation are uniformly distributed, the node positions are all represented by node numbers to save parameter bits, numbered sequentially according to the "orbit number–intra-orbit satellite number" order. Taking a 4×4 satellite network as an example, the numbering rule is shown in Fig. 2.

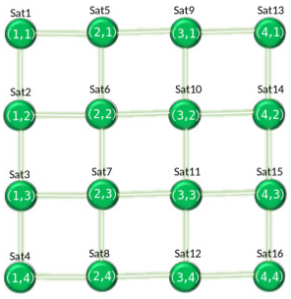

Fig. 2. Satellite-numbering rule.

The node-state parameters comprise link topology and link load: the link topology is represented in binary form to indicate connectivity (1 reachable, 0 unreachable); the link load is determined by the queue backlog, marked as congested (1) when queue occupancy exceeds 50% of capacity and 0 otherwise. Combining the above information yields a unified input-packet format, as shown in Fig. 3. Taking 66 satellites each with 4 interfaces as an example, every forwarding operation requires the source node ID, current node ID, and destination node ID (2 bytes each), as well as the per-interface load (1 byte each) and neighbor node IDs (2 bytes each) of every satellite, repeated across the whole network.

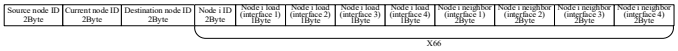

Fig. 3. Input-packet format of the load-balancing inter-satellite routing algorithm.

C. Action Space and Output Mapping

The algorithm output is the next-hop node ID. The DQN first computes the next-hop forwarding direction, which is then mapped to the concrete next-hop node ID through the forwarding-direction–decision-value lookup table and output to EXata. Taking node 6 in Fig. 2 as an example, the mapping between the four forwarding directions and the next-hop IDs is given in Table 1. This mapping converts the abstract forwarding direction directly into a node number recognizable by the simulation system and the satellite equipment, achieving seamless integration from algorithm decision to actual forwarding.

TABLE I. Forwarding-direction–decision-value (for node 6)

Decision value	***Forwarding-direction meaning***	***Next-hop ID***
0	Forward to preceding satellite in the same orbit	5
1	Forward to following satellite in the same orbit	7
2	Forward to co-located satellite in the preceding orbital plane	2
3	Forward to co-located satellite in the following orbital plane	10

D. Reward-Function Design

The reward-feedback mechanism follows three principles: the larger the Euclidean distance to the destination node after forwarding, the lower the single-step reward; the higher the load of the next-hop node, the lower the reward; and if forwarding reaches the destination node, the reward is the highest. Accordingly, the reward-computation formula is designed as the piecewise function in equation (1):

$$R=\begin{cases} R_s, & \text{if the destination node is reached} \\ -R_c, & \text{if a congested node is reached} \\ -a \cdot \mathrm{Dis}(dst, cur)^2, & \text{otherwise} \end{cases} \quad (1)$$

where R is the single-step reward; R_s and R_c are large constants; a is the parameter weight; and *Dis(dst, cur)* is the Euclidean distance from the next-hop node to the destination node.

E. Double-DQN Training Algorithm

The Double DQN training algorithm designed in this paper follows a "decision–feedback–learning" thread and mainly comprises three processes—action selection, reward feedback, and network-parameter update—whose overall structure is shown in Fig. 4.

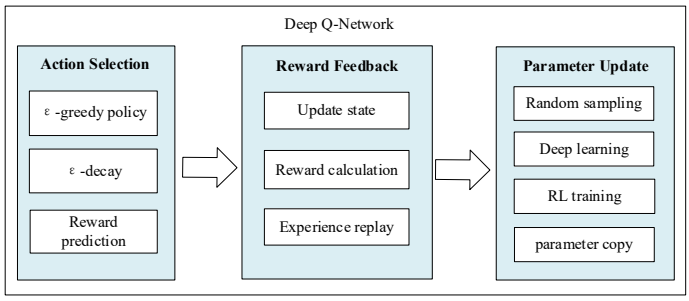

Fig. 4. Main flow of the DQN algorithm.

As shown in Fig. 4, the algorithm consists of three linked functional modules. The action-selection module adopts the ε-greedy strategy: it performs random exploration with probability ε or picks the action with the maximum estimated reward with probability 1−ε, while decaying ε and predicting action rewards. The reward-feedback module updates the network state after action execution, calculates instant rewards and stores experience tuples into the experience pool. When the experience pool reaches the preset capacity, the network-parameter-update module conducts random sampling, deep reinforcement learning training and target network parameter copying to update the model. These three modules run cyclically, and the model gradually converges to the optimal forwarding policy via constant interaction with the environment.

1) Action Selection (ε-Greedy Strategy)

The ε-greedy strategy is used for action decision-making: with probability (1−ε) the action with the highest estimated reward is selected, and with probability ε an action is explored randomly; ε is decayed after each successful routing, so that as training proceeds the model selects the existing optimal decision with higher probability, thereby balancing exploration and exploitation, as shown in Fig. 5.

The algorithm generates a random number r and compares it with the exploration rate ε. If r > ε, the model selects the action with the highest estimated reward for exploitation; otherwise, it picks an action randomly for exploration. After action execution, ε decays if the packet reaches the destination node, making subsequent decisions rely more on the learned policy. This mechanism ensures sufficient exploration at the early training stage and gradual convergence to a stable, efficient forwarding policy in later phases, avoiding inadequate exploration and premature convergence.

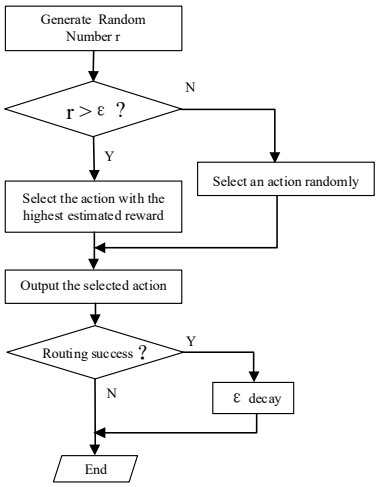

Fig. 5. Schematic of the ε-greedy action-selection strategy.

2) Reward Feedback and Experience Storage

The environment updates the packet position according to each forwarding decision (as shown in Fig. 6) and computes the single-step reward according to equation (1). The four-tuple of the original state, decision action, reward value, and next state $\langle s_i, a_i, r_i, s_i' \rangle$ is then stored in the experience pool (as shown in Fig. 7) for subsequent random sampling and offline learning.

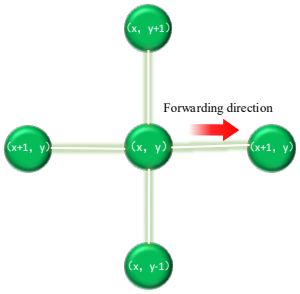

Fig. 6. Schematic of position update.

As shown in Fig. 6, the packet currently resides at node (x, y), whose four neighbors are (x, y−1) and (x, y+1) in the same-orbit direction and (x−1, y) and (x+1, y) in the adjacent orbital planes. When the agent outputs a particular forwarding direction (illustrated by the arrow pointing to (x+1, y) in the figure), the packet migrates from the current node to the neighbor node corresponding to that direction, which becomes the new current position entering the next-hop decision, thereby completing one state update.

<s1,a1,r1,s1'>
< s2,a2,r2,s2'>
< s3,a3,r3,s3'>
...

Fig. 7. Experience storage.

As shown in Fig. 7, the experience pool stores transition samples from agent-environment interactions. Each sample tuple consists of the original state s_i, action a_i, reward r_i and next state s_i'. These samples are accumulated chronologically for random sampling and network parameter updates. Random sampling breaks temporal correlation among samples, thus boosting learning stability and sample utilization efficiency.

3) Network-Parameter Update (DDQN)

The capacity of the experience pool is used as the parameter-update period: an update is performed once the number of samples in the pool reaches the capacity, and the target network draws a batch of samples at random from the experience pool for learning each time (as shown in Fig. 8).

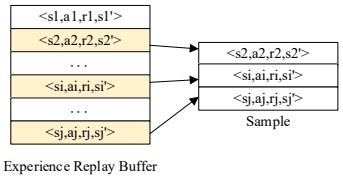

Fig. 8. Random sampling.

DDQN is adopted for parameter update to suppress Q-value overestimation. The algorithm uses a main value-function network and an auxiliary target network. For each sample, we input state s(t+1) into the value Q-network to select the action a′ with the maximum Q-value via equation (2):

$$a' = argmax(Q(s_{t+1}, \theta)) \tag{2}$$

where Q(s(t+1), θ) represents Q-values of all corresponding actions. We then substitute s(t+1) and a′ into the target network to calculate the target Q-value *Q_new* as shown in equation (3):

$$Q_{new} = r + \gamma \cdot (1 - done) * Q_{target}(s_{t+1}, a', \theta') \tag{3}$$

Here r is the instant reward, *done* marks whether the destination node is reached, γ is the discount factor (generally 0.9), and θ′ stands for target network parameters. The loss is defined as the squared error between the network output Q-value and *Q_new* (equation (4)), and back propagation is applied to update parameters of the value-function network:

$$Loss = (Q(s, a, \theta) - Q_{new})^2 \tag{4}$$

Periodically, the value-function network parameters are copied to the target network at a fixed update interval. The complete training process is illustrated in Fig. 9.

IV. Experiments and Result Analysis

A. Simulation Platform and Scenario

The experiments adopt an HLA/RTI distributed simulation platform with five modules: simulation control, EXata protocol simulation, STK9 3D visualization, DDQN-based intelligent routing and platform RTI. All modules work as HLA federation members via standard RTI interfaces. The protocol simulation unit communicates with the routing unit through sockets to exchange network states and routing decisions, forming a complete simulation closed loop. For the protocol stack, CBR traffic runs at the application layer, UDP at the transport layer. The network layer adopts inter-satellite load balancing routing and IP-tunnel virtual access routing, and the physical layer emulates point-to-point laser communication.

The simulation includes space and ground segments. The space segment supports 40–100 satellites, and 66 satellites are deployed in this test. The ground segment is configured with 2 ground stations and 10 terminals. Detailed space network parameters are given in Table 2, and the EXata simulation scenario is shown in Fig. 10.

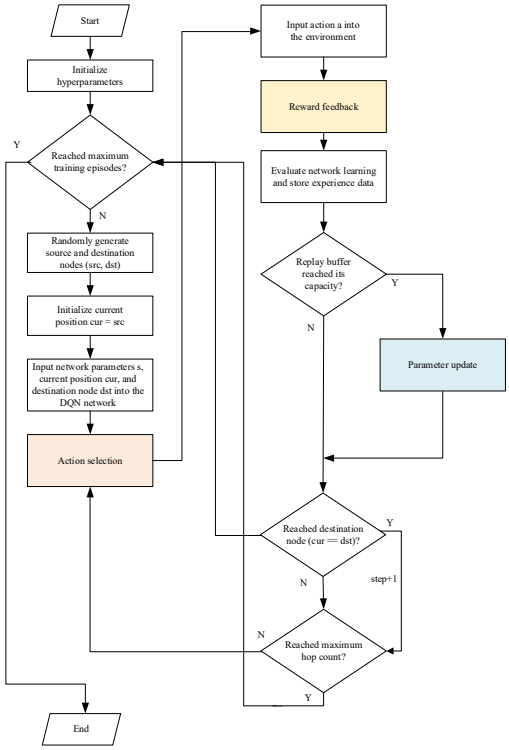

Fig. 9. Overall training flowchart of the algorithm.

TABLE II. Network-structure parameters of the LEO satellite constellation.

Parameter category	***Value***
Number of satellites	*66*
Orbital altitude (km)	780
Number of orbital planes	11
Satellites per orbit	6
Orbital inclination	86.4°

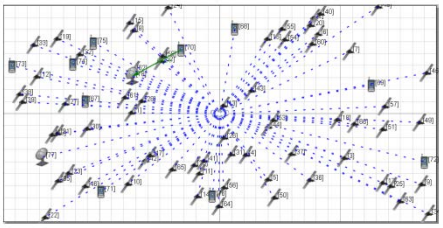

Fig. 10. EXata software simulation scenario.

B. Evaluation Metrics

The following metrics are used to evaluate routing performance. The routing-convergence time is the time required from a link on/off change to the completion of network-wide topology synchronization, as shown in equation (5):

$$T_{collect} = T_r - T_0 \tag{5}$$

$T_{collect}$ denotes the single route convergence time, T_r denotes the network-wide route convergence time instant, T_0 denotes the time instant when the topology change occurs.

The normalized routing overhead is the ratio of the total number of routing-signaling bytes per unit time to the network-wide design capacity, as shown in equation (6):

$$cost_{rp} = \frac{C_{signaling-p}}{C_{Design}} \times 100\% \quad (6)$$

$cost_{rp}$ is defined as the normalized routing overhead, which is determined by the total byte count of routing signaling $C_{signaling-p}$ and the network design capacity C_{Design}.

The end-to-end delay $Delay_{endtoend}$ is the time from when a flow is generated at the application layer of the source node to when it is received at the application layer of the destination node, as shown in equation (7):

$$Delay_{\text{end-to-end}} = T_{\text{receive}} - T_{\text{send}} \quad (7)$$

$T_{receive}$ denotes service receive time at destination application layer, T_{send} denotes service generation time at source node.

The ground-station downlink traffic difference is the ratio of the difference in the total downlink traffic received by different ground stations to the total traffic, as shown in equation (8):

$$\text{diff}_{i,j} = \frac{|C_{Gi} - C_{Gj}|}{max\{C_{Gi},\ C_{Gj}\}} \times 100\% \quad (8)$$

$\text{diff}_{i,j}$ quantifies the downlink traffic imbalance between ground stations i and j, C_{Gi} and C_{Gj} are the total traffic received by each station.

The packet-loss rate is the ratio of the total number of packets received at the destination to the total number of packets sent at the source, as shown in equation (9):

$$P_{loss} = \frac{N_{packet\_receive}}{N_{packet\_send}} \times 100\% \quad (9)$$

C. Result Analysis

Based on the 66-satellite simulation scenario above, a preliminary simulation of the proposed method is conducted, and the results of all metrics are summarized in Table 3.

TABLE III. SIMULATION PERFORMANCE-METRIC RESULTS.

Performance metric	***Result***	***Condition / remark***
Average transmission hops	8 hops	66-satellite scenario
Maximum routing-convergence time	≈ 270 ms	Link maintenance keeps convergence < 1 s
Normalized routing overhead	≈ 4.0%	Signaling bytes / design capacity
End-to-end delay	≈ 0.96 s	Transmission delay + queuing delay
Ground-station downlink traffic difference	≈ 10%	Traffic evenly split among ground stations
Packet-loss rate (20% satellite failure)	≈ 0.3%	Under 20% satellite-failure condition

Experimental results show that the proposed method achieves an average path length of 8 hops. Supported by the laser link physical layer feedback mechanism, its maximum routing convergence time is around 270 ms (less than 1 s), and the normalized routing signaling overhead is approximately 4.0%. The end-to-end delay is about 0.96 s, mainly composed of transmission and queuing delay. Benefiting from the distance-load joint reward and satellite-ground load balancing strategy, service flows are evenly distributed across ground stations, with a downlink traffic difference of roughly 10%. In the 20% satellite failure scenario, triggered flooding ensures fast topology synchronization and DDQN enables adaptive congestion avoidance, keeping the packet loss rate at about 0.3%. The results prove that the dual-network structure and DDQN can suppress Q-value overestimation, speed up convergence and stabilize decisions. All indicators satisfy the project requirements.

V. CONCLUSION AND FUTURE WORK

We propose a load-balancing inter-satellite routing scheme for LEO networks based on Double DQN. Experimental results verify its favorable performance for space-based IoT, LEO routing and integrated satellite-terrestrial networks. Nevertheless, the existing method adopts a binary hard threshold of 50% buffer occupancy for congestion judgment. Despite simplicity and low signaling overhead, this criterion easily triggers state oscillation under load fluctuation around the threshold and bursty traffic, and fails to differentiate lightly loaded and near-saturated links. Future refinement will replace the binary indicator with continuous or multi-level refined congestion metrics, combined with load-smoothing reward functions and critical hysteresis bands to mitigate oscillation and improve routing stability and end-to-end latency. Further research will compare various congestion characterization strategies and explore algorithm scalability for large-scale constellations as well as the integration of multi-agent cooperation and graph neural networks.

REFERENCES

[1] Heng Du, Ligang Cong, Qingyun Liang, and Shuyu Wang. 2026. SDRP-DQL: A satellite distributed routing protocol based on double deep Q-learning. Computer Networks (2026). https://www.sciencedirect.com/science/article/abs/pii/S1389128626000800.

[2] Chi Han, Wei Xiong, and Ronghuan Yu. 2024. Deep reinforcement learning-based multipath routing for LEO megaconstellation networks. Electronics 13, 15 (2024), 3054. https://doi.org/10.3390/electronics13153054.

[3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. 2015. Human-level control through deep reinforcement learning. Nature 518, 7540 (2015), 529–533. https://doi.org/10.1038/nature14236.

[4] Yan Chen, Huan Cao, Longhe Wang, Daojin Chen, Zifan Liu, Yiqing Zhou, and Jinglin Shi. 2025. Deep reinforcement learning-based routing method for low earth orbit mega-constellation satellite networks with service function constraints. Sensors 25, 4 (2025), 1232. https://doi.org/10.3390/s25041232.

[5] Federico Lozano-Cuadra and Beatriz Soret. 2024. Multi-agent deep reinforcement learning for distributed satellite routing. arXiv:2402.17666. https://doi.org/10.48550/arXiv.2402.17666.

[6] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double Q-learning. In Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI'16). 2094–2100. https://doi.org/10.1609/aaai.v30i1.10295.

[7] Manuel M. H. Roth, Anupama Hegde, Thomas Delamotte, and Andreas Knopp. 2024. Shaping rewards, shaping routes: On multi-agent deep Q-networks for routing in satellite constellation networks. arXiv:2408.01979. https://doi.org/10.48550/arXiv.2408.01979.

[8] Yulan Gao, Ziqiang Ye, and Han Yu. 2024. Cost-efficient computation offloading in SAGIN: A deep reinforcement learning and perception-aided approach. arXiv:2407.05571. https://doi.org/10.48550/arXiv.2407.05571.

[9] Ben Wang, Chang Liu, Peng Liu, Yi Sun, and Yang Yang. 2025. Adaptive satellite network bandwidth allocation method based on sequential DDQN model. In Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-981-96-6459-7_18.